

Exploring Flow Matching and Anomaly Detection Documentation

Contents

1	Introduction & Setting	2
2	Mathematical Framework	3
3	Files & Code details	6
3.1	dataset.py	6
3.2	paths.py	7
3.3	vector_field_unet.py	7
3.4	train_unet.py	8
3.5	solve.py	9
3.6	test_anomalies.py	9
3.7	evaluate_roc.py	10
3.8	sweep_time.py	10
4	Results	11
4.1	Score distributions and thresholds	12
4.2	Dependence on the smoothing parameter	13
4.3	Number of Monte Carlo samples	14
4.4	Spatial decomposition of the score	14
4.5	Dependence on the time variable	15

1 Introduction & Setting

This project started as an implementation of conditional flow matching, following the TU Berlin course *Mathematical Foundations of Machine Learning* (summarised in Section 2). The model is a transport map: a vector field $\xi_\theta(y, t)$ whose flow carries a Gaussian source $\nu = \mathcal{N}(0, I_d)$ at $t = 0$ towards the data distribution μ at $t = 1$, so that integrating the ODE from a noise sample produces a new sample from μ . As data we use MNIST, 28×28 greyscale images of handwritten digits, restricted to the digit 0.

Once this worked as a generator, anomaly detection offered a concrete test of what such a model actually learns. Training on a single class makes this a one-class problem: only "good" examples are available, and any other digit counts as an anomaly. The first score is the one the generative interpretation suggests directly. The ODE setting gives the log-density of an observation explicitly, via Liouville's formula and the change-of-variables formula, so an image can be scored by its negative log-density under the learned model - how implausible it is as a zero. This score however did not perform well.

This motivates a second experiment. We reuse the conditional flow matching objective itself as an anomaly score S_{CFM} , evaluating the frozen vector field on a new observation. It performs substantially better, reaching an AUC of 0.985 - but it does not use the generative structure of the learned flow at all: neither the ODE integration nor the density enters its computation.

That raises the question of which part of the learned representation carries the signal. S_{CFM} averages over $t \in [0, 1]$; evaluating it at fixed t instead shows that the separation is strongest around $t \in [0.2, 0.3]$, while at the endpoint $t = 0$ the score reduces to a simple distance-to-mean baseline.

The overall pipeline is:

1. Restrict the training data to digit 0 and flatten each image to a vector in $\mathbb{R}^d$, $d = 784$.
2. Construct Gaussian conditional probability paths from the source $\nu = \mathcal{N}(0, I_d)$ to the training samples and derive their conditional vector fields.
3. Train a U-Net $\xi_\theta(y, t)$ with the conditional flow matching objective.
4. Integrate the resulting ODE with a second-order Runge–Kutta scheme: forward from a sample of ν to generate an image, backward from an observed image to recover its latent variable. In the backward direction, accumulating the divergence along the path with a Hutchinson trace estimate additionally gives the model density.
5. Score new images, compare the two scores, and sweep the flow time t .

The document records this exploration rather than a finished result: the structure follows the order in which the questions came up, and it is written to make the project reproducible for me and others, not as a rigorous treatment.

2 Mathematical Framework

We model a time-continuous normalizing flow as the solution of the ODE

$$\frac{d}{dt}y(t) = \xi(y(t), t), \quad y(0) = y_0,$$

with $\xi : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$, restricting the time interval to $[0, 1]$ throughout. Let $\phi_{s,t}$ denote the flow map sending y_s to y_t . Under suitable regularity assumptions on ξ , the ODE admits a unique flow of invertible maps, with

$$\phi_{t,s} = \phi_{s,t}^{-1}.$$

We fix a source measure $\nu = \mathcal{N}(0, I_d)$ with density

$$f_\nu(z) = (2\pi)^{-d/2} \exp\left(-\frac{1}{2}z^\top z\right),$$

and a target measure μ , the distribution of good products, whose density f_μ is unknown and available only through samples.

We parametrize the vector field $\xi = \xi_\theta$ by a neural network. The goal is to learn ξ_θ such that the flow map generated by it satisfies

$$\phi_{0,1*}\nu \approx \mu,$$

that is, $\phi_{0,1}$ is a transport map from ν to μ . Read forward, it turns source samples into products; read backward, $\phi_{1,0}$ maps an observed product to its latent variable and normalizes μ back to the Gaussian ν , which is the direction used for density evaluation.

Write $\mu_\theta = \phi_{0,1*}\nu$ for the distribution the model actually represents and f_θ for its density, so that $f_\mu \approx f_\theta$ if the flow is learned well. The change of variables formula gives

$$f_\mu(x) \approx f_\theta(x) = f_\nu\left(\phi_{0,1}^{-1}(x)\right) \left|\det D\phi_{0,1}^{-1}(x)\right|,$$

and abbreviating the latent variable of x by $z = \phi_{1,0}(x) = \phi_{0,1}^{-1}(x)$,

$$\log f_\theta(x) = \log f_\nu(z) + \log |\det D\phi_{1,0}(x)|.$$

In the ODE setting, the log-determinant takes a particularly simple form. By Liouville's formula,

$$\log |\det D\phi_{s,t}(y_s)| = \int_s^t \operatorname{div}_y \xi_\theta(\phi_{s,\tau}(y_s), \tau) d\tau.$$

Applying this we obtain

$$\log f_\mu(x) \approx \log f_\theta(x) = \log f_\nu(z) - \int_0^1 \operatorname{div}_y \xi_\theta(y_t, t) dt. \quad (1)$$

The log-density of the standard multivariate Gaussian is

$$\log f_\nu(z) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} z^\top z. \quad (2)$$

It remains to determine θ . We regress ξ_θ onto a target vector field,

$$L_{\text{FM}}(\xi_\theta, \xi) = \mathbb{E} \left[|\xi_\theta(Y, T) - \xi(Y, T)|^2 \right],$$

where $T \sim U([0, 1])$ and $Y \sim f_t(y) dy$ along a probability path interpolating between ν and μ . The difficulty is that neither the marginal probability path f_t nor its associated vector field ξ is known, since the target distribution is only available through samples.

Conditional flow matching circumvents this difficulty by introducing a conditional probability path for each data point $x \sim \mu$. For fixed x , we construct a conditional path $f_t^\sigma(y | x)$, which describes how the source distribution is transported towards the individual data point x . The parameter $\sigma > 0$ controls the width of the conditional distribution at the endpoint and thus acts as a regularization parameter. We require the conditional paths to start from the common source distribution,

$$f_0^\sigma(y | x) = f_\nu(y),$$

and to become concentrated around x at $t = 1$. Averaging the conditional paths over the data distribution gives the marginal path

$$f_t^\sigma(y) = \int_{\mathbb{R}^d} f_t^\sigma(y | x) f_\mu(x) dx,$$

whose endpoint approaches the data distribution as $\sigma \rightarrow 0$,

$$f_1^\sigma(y) \longrightarrow f_\mu(y).$$

Suppose that, for every fixed x , the conditional vector field $\xi(y, t | x)$ transports the corresponding conditional probability path, i.e. it satisfies the continuity equation

$$\frac{\partial}{\partial t} f_t^\sigma(y | x) = -\text{div}_y (\xi(y, t | x) f_t^\sigma(y | x))$$

with x treated as a fixed parameter. Then the conditional vector fields can be averaged to obtain a vector field for the marginal path,

$$\xi(y, t) = \frac{1}{f_t^\sigma(y)} \int_{\mathbb{R}^d} \xi(y, t | x) f_t^\sigma(y | x) f_\mu(x) dx, \quad (3)$$

and this ξ satisfies the continuity equation for f_t^σ . Thus, although the marginal vector field is not directly available, it can be represented in terms of conditional vector fields that are constructed from individual data samples. Both $\xi(y, t | x)$ and $\xi(y, t)$ depend on σ through the conditional path; we suppress this in the notation and keep the superscript only on the densities, where the limit $\sigma \rightarrow 0$ is discussed.

Since ξ satisfies the continuity equation for f_t^σ with initial condition $f_0^\sigma = f_\nu$, the flow it generates transports ν along the marginal path and therefore reaches its endpoint (the target distribution only up to the smoothing introduced by σ),

$$\phi_{0,1*} \nu = f_1^\sigma(y) dy.$$

This representation also gives a tractable training objective. Instead of regressing the neural vector field onto the unknown marginal field, we regress it onto the conditional field:

$$L_{\text{CFM}}(\xi_\theta, \xi(\cdot, \cdot | \cdot)) = \mathbb{E} \left[|\xi_\theta(Y, T) - \xi(Y, T | X)|^2 \right], \quad (4)$$

where $X \sim \mu$, $T \sim U([0, 1])$, and $Y \sim f_T^\sigma(\cdot | X)$. Drawing n i.i.d. triples (X_j, T_j, Y_j) in this way gives the empirical objective

$$\hat{L}_{\text{CFM}} = \frac{1}{n} \sum_{j=1}^n |\xi_\theta(Y_j, T_j) - \xi(Y_j, T_j | X_j)|^2, \quad (5)$$

which is a standard regression problem. The difference $L_{\text{FM}} - L_{\text{CFM}}$ does not depend on ξ_θ , so both objectives have the same gradients with respect to θ and the same minimizers (i.e. $\hat{L}_{\text{CFM}}$ is an unbiased empirical risk function of L_{FM}).

As is common in practice, we use Gaussian probability paths,

$$f_t^\sigma(y | x) dy = \mathcal{N}(m_t(x), \sigma_t^2 I_d)(dy),$$

where

$$m_0(x) = 0, \quad m_1(x) = x, \quad \sigma_0 = 1, \quad \sigma_1 = \sigma,$$

so that the path starts at the source distribution $\nu = \mathcal{N}(0, I_d)$ and ends in a Gaussian concentrated around the data point x . The marginal endpoint distribution is then the convolution

$$f_1^\sigma(y) = \int_{\mathbb{R}^d} f_1^\sigma(y | x) f_\mu(x) dx = (f_\mu * f_{\mathcal{N}(0, \sigma^2 I_d)})(y),$$

which converges to $f_\mu(y)$ as $\sigma \rightarrow 0$ under suitable conditions.

The Gaussian conditional path is generated by the affine transformation

$$\phi_{0,t}(y | x) = \sigma_t y + m_t(x), \quad (6)$$

since $\nu = \mathcal{N}(0, I_d)$ and therefore

$$\phi_{0,t}(\cdot | x)_* \nu = \mathcal{N}(m_t(x), \sigma_t^2 I_d).$$

To obtain the associated conditional transport vector field, we require

$$\frac{d}{dt} \phi_{0,t}(y | x) = \xi(\phi_{0,t}(y | x), t | x),$$

which, with an affine ansatz for $\xi(\cdot, t | x)$ and comparison of coefficients, gives

$$\xi(y, t | x) = \frac{\sigma_t'}{\sigma_t} (y - m_t(x)) + m_t'(x).$$

Choosing the concrete schedules

$$\sigma_t = (1 - t) + t\sigma, \quad m_t(x) = tx, \quad (7)$$

we obtain

$$\sigma'_t = \sigma - 1, \quad m'_t(x) = x,$$

and therefore the explicit conditional vector field

$$\xi(y, t \mid x) = x + \frac{\sigma - 1}{\sigma_t} (y - tx). \quad (8)$$

Sampling Y from the conditional path amounts to the reparametrization $Y = tx + \sigma_t \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I_d)$, and inserting this into (8) makes the regression target particularly transparent:

$$\xi(Y, t \mid x) = x + (\sigma - 1)\varepsilon. \quad (9)$$

The target is thus an affine combination of the data point and the noise that was used to perturb it, with t entering only through the input Y of the network.

The resulting generative process is illustrated in Figure 1.

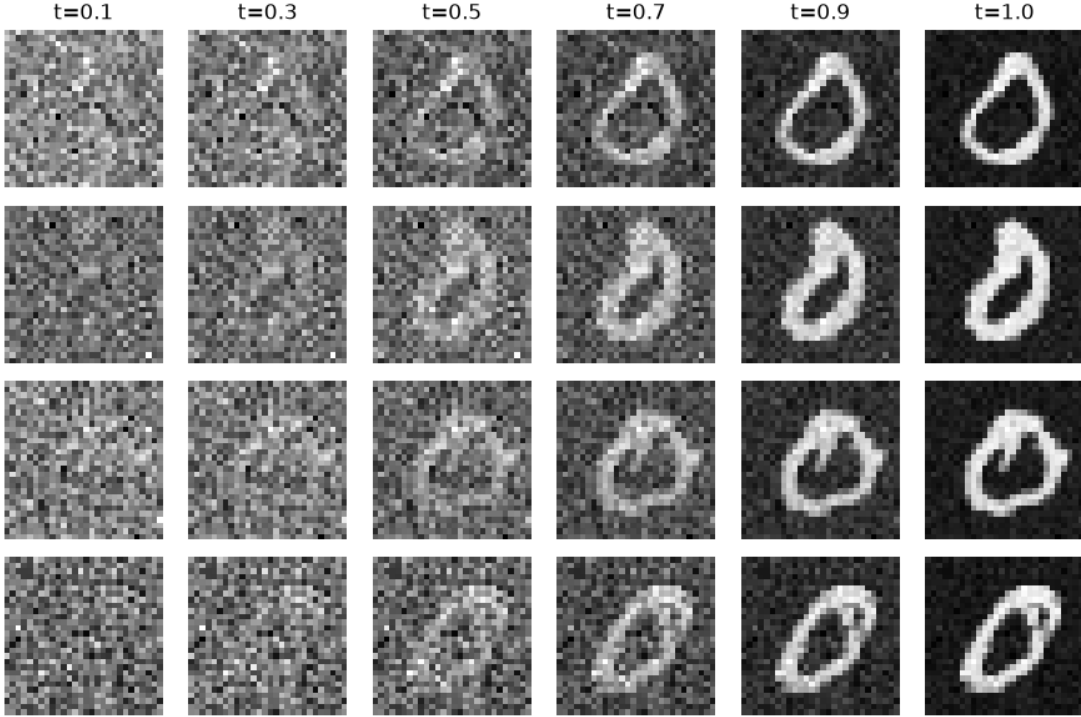


Figure 1: Forward sampling with the learned flow.

3 Files & Code details

3.1 dataset.py

`MNISTDigitDataset` loads the MNIST dataset and converts each image to a `PyTorch` tensor, rescaling the pixel values of each image from $[0, 255]$ to $[0, 1]$ and then to $[-1, 1]$. The dataset is then

filtered to a single digit class, which serves as the set of "good" samples (which are zeros in our case) for the proof of concept. The filtered images are stacked into a single tensor and flattened from 28×28 to vectors in $\mathbb{R}^d$, $d = 784$, matching the generic notation $x, y, z \in \mathbb{R}^d$ used in Sections 2–3. Finally, `__len__` and `__getitem__` are implemented so that a `DataLoader` can draw mini-batches $X \sim \mu$ during training.

3.2 paths.py

This file implements the schedules (7) and the resulting conditional vector field (8). `sigma_t` evaluates σ_t from (7). `sample_conditional_path` draws a sample Y from the conditional path by applying (6) to a source sample $\varepsilon \sim \mathcal{N}(0, I_d)$, given a batch X , time points T , and the regularization parameter σ ; it returns both Y and ε . `conditional_vector_field` evaluates (8). Both functions are used to construct training targets during optimization and as ground truth in the diagnostic and testing scripts.

3.3 vector_field_unet.py

The neural vector field ξ_θ is parameterized by a U-Net adapted to the 28×28 MNIST images, as shown in Figure 2. The input $y \in \mathbb{R}^{784}$ is reshaped to an image and concatenated with a second channel containing the time variable t at every spatial location. The encoder uses two convolutional downsampling stages, reducing the spatial resolution from 28×28 to 7×7 while increasing the number of feature channels from 32 to 128. The decoder restores the original resolution using nearest-neighbor upsampling and skip connections from the corresponding encoder stages. All convolutional blocks consist of two 3×3 convolutions with padding 1 and SiLU activations. The final 3×3 convolution maps the 32 decoder channels to a single output channel, which is flattened to obtain the predicted vector field $\xi_\theta(y, t) \in \mathbb{R}^{784}$.

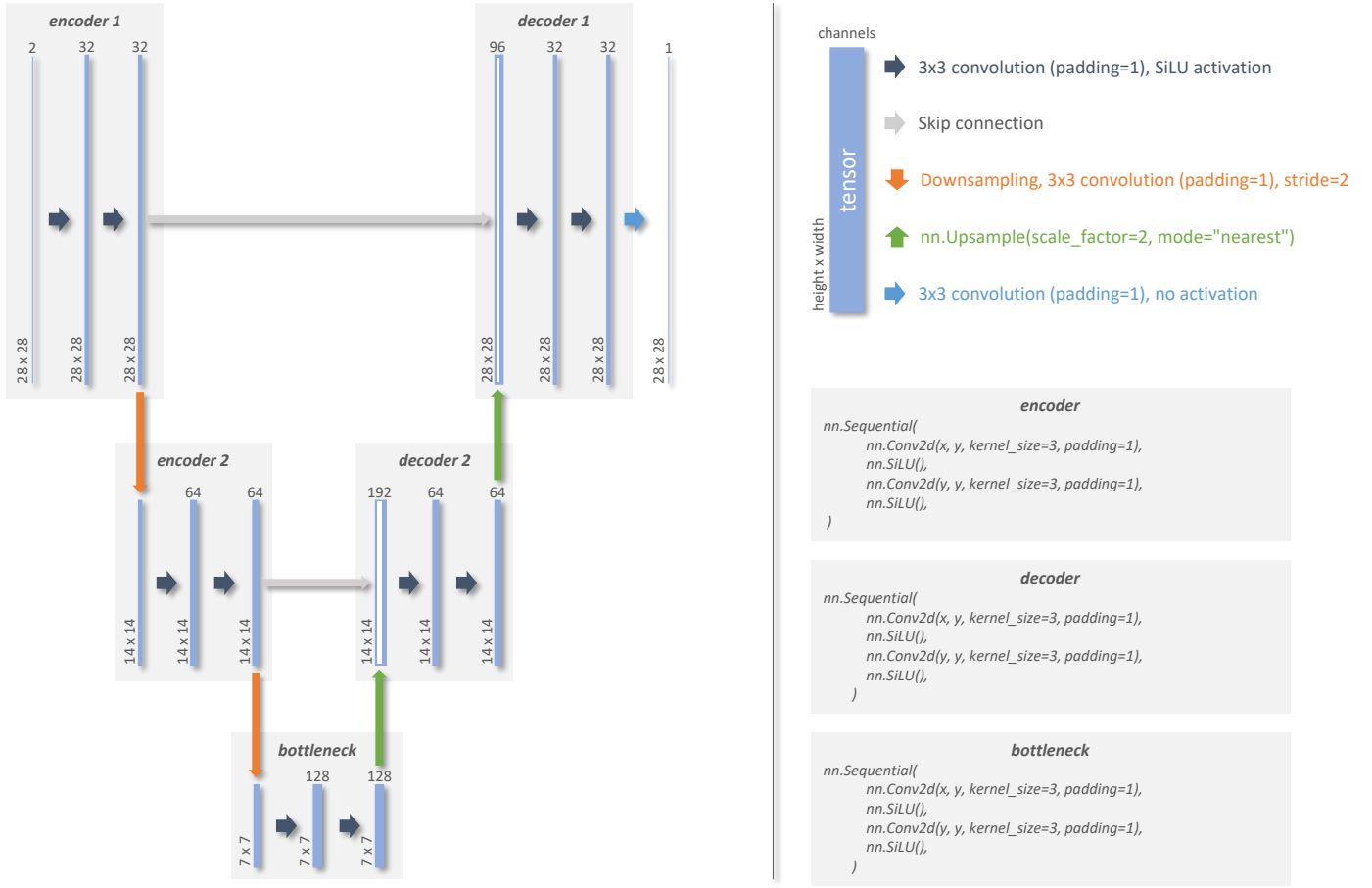


Figure 2: U-Net architecture used to parameterize the neural vector field ξ_θ .

3.4 train_unet.py

This file implements the training loop that minimizes L_{CFM} (4), approximating the expectation by Monte Carlo over mini-batches of n i.i.d. triples (X_j, T_j, Y_j) . For each mini-batch X drawn from the dataset μ , an independent time $T_j \sim U([0, 1])$ is drawn per sample, and Y_j is obtained via `sample_conditional_path` as in (6). The target $\xi(Y_j, T_j \mid X_j)$ is evaluated via `conditional_vector_field` (8), and the network prediction $\xi_\theta(Y_j, T_j)$ is obtained via a forward pass through `VectorFieldUNet`. The implemented loss is

$$L = \frac{1}{nd} \sum_{j=1}^n \|\xi_\theta(Y_j, T_j) - \xi(Y_j, T_j \mid X_j)\|_2^2,$$

scaled by $1/d$ due to averaging (rather than summing) over the $d = 784$ output dimensions (affects only the step size, which is adaptive in Adam anyways, not the minimizer θ^*). Gradients are computed via backpropagation and applied using the Adam optimizer. Training runs for a fixed number of epochs over the full dataset of digit-0 images, and the resulting weights θ^* are saved to `checkpoints/unet_vector_field.pt` for use in the evaluation and testing scripts.

3.5 solve.py

This file implements numerical integration of the ODE via the second-order Runge-Kutta scheme, together with the divergence integral needed for (1).

`hutchinson_divergence` estimates $\text{div}_y \xi$ at a given point without forming the full Jacobian. Using a Rademacher vector ε (entries ± 1) and the identity $\mathbb{E}[\varepsilon^\top A \varepsilon] = \text{tr}(A)$, a single vector-Jacobian product $\varepsilon^\top D_y \xi$, obtained via one backward pass, gives an unbiased Monte Carlo estimate of the trace $\text{tr}(D_y \xi) = \text{div}_y \xi$. Averaging over `num_probes` independent draws of ε reduces the variance of this estimate.

`solve` integrates forward from $t = 0$ to $t = 1$, starting at a source sample z_0 , using the RK2 update

```
k1 = model(y, t)
y_mid = y + (h / 2) * k1
k2 = model(y_mid, t + h / 2)
y = y + h * k2
```

at each step of size h . If `compute_likelihood=True`, the divergence at each midpoint is accumulated via `hutchinson_divergence`, approximating $\int_0^1 \text{div}_y \xi(y_t, t) dt$, and combined with (2) evaluated at z_0 to return $\log p_\theta(x)$ via (1).

`solve_backward` integrates the same ODE in the reverse direction, from a given image x_1 at $t = 1$ back to $t = 0$, recovering $z = \phi_{1,0}(x_1)$. The RK2 update signs are flipped to reflect integration against the direction of ξ ; the accumulated divergence integral itself needs no sign correction, since it still sums $\text{div}_y \xi$ over the same interval $[0, 1]$, only visited in reverse order. Here (2) is evaluated at the recovered z , giving $\log p_\theta(x_1)$ for an observed image x_1 — this is the direction used for anomaly scoring, whereas `solve` is used to check the self-consistency of freshly generated samples.

3.6 test_anomalies.py

This file evaluates the anomaly score on the test split. The score reuses the training objective (4) with the weights θ^* frozen: for a single image x ,

$$S_{\text{CFM}}(x) = \mathbb{E}_{T, \varepsilon} \left[\frac{1}{d} \|\xi_{\theta^*}(Y, T) - \xi(Y, T \mid x)\|_2^2 \right], \quad T \sim U([0, 1]), \quad Y \sim f_T^\sigma(\cdot \mid x), \quad (10)$$

estimated by Monte Carlo over `num_samples` independent draws of the pair (T, ε) ; the additional $1/d$ compared to (4) only rescales the score, not its ranking across images, since it is applied identically to every x .

By the reparametrization (9), the target inside the norm is $x + (\sigma - 1)\varepsilon$, so the score measures how well the frozen field predicts the combination of data point and perturbation that defines the conditional target. Since ξ_{θ^*} was fitted to the conditional vector fields of the normal class only, this works well for images from that class and less well for others.

`score_cfm_loss` implements the estimator, averaging over the d output dimensions so that one score per image is returned. It takes an optional argument `t_fixed`: by default T is drawn uniformly as

in (10), and if a value is passed, that value is used for every draw instead. The latter is used in Section 3.8. `evaluate_digit` computes the scores for the first n test images of a given digit, and `test_anomalies` iterates over all ten digits.

For threshold-based evaluation, decision thresholds are computed directly from the quantiles of the digit-0 test score distribution. Consequently, the resulting false-positive rates are exact by construction rather than calibrated on a separate validation set. `print_separation` takes a list of quantiles of the digit-0 score distribution as thresholds, where the quantile 1.0 corresponds to the maximum observed score; for each anomalous class it counts the images falling below each threshold, i.e. the false negatives. Sweeping over several quantiles makes the trade-off between false positives and false negatives explicit: a lower quantile lowers the threshold and thus reduces false negatives, at the cost of rejecting a larger fraction of normal images.

For comparison, a second score is computed directly from the learned density,

$$S_{\text{NLL}}(x) = -\log f_{\theta}(x),$$

evaluated via (1) using `solve_backward`. The sign is chosen so that, as for S_{CFM} , larger values indicate less typical images (`score_likelihood` implements this). This was the original idea for detecting anomalies; its poor performance, reported in Section 4, is what motivated S_{CFM} above.

3.7 `evaluate_roc.py`

While Section 3.6 evaluates the score at a small number of fixed thresholds, this file characterizes it across all thresholds simultaneously. Labelling anomalies as positives and normal images as negatives, a threshold τ induces the classification $S_{\text{CFM}}(x) \geq \tau \Rightarrow \text{anomaly}$, with the two rates

$$\text{TPR}(\tau) = \frac{\text{TP}(\tau)}{\text{TP}(\tau) + \text{FN}(\tau)}, \quad \text{FPR}(\tau) = \frac{\text{FP}(\tau)}{\text{FP}(\tau) + \text{TN}(\tau)},$$

i.e. the fraction of anomalies correctly detected and the fraction of normal images wrongly rejected. The ROC curve is the parametric curve $\tau \mapsto (\text{FPR}(\tau), \text{TPR}(\tau))$ obtained by sweeping τ over all observed scores, and the area under it (the AUC) summarizes the separation in a single threshold-free number. It admits the interpretation

$$\text{AUC} = \mathbb{P}(S(X_{\text{anomaly}}) > S(X_{\text{normal}})),$$

the probability that a randomly drawn anomaly receives a higher score than a randomly drawn normal image.

Both scores are computed for the digit-0 test images and the pooled foreign digits, and passed with the corresponding labels to `roc_curve` and `roc_auc_score` from `scikit-learn`. The resulting ROC curves are shown in Section 4.

3.8 `sweep_time.py`

The score (10) averages over $T \sim U([0, 1])$. This file evaluates it at fixed values of t instead, to see how much each time contributes to the separation. No retraining is involved; the same checkpoint is used throughout.

`_auc_for` computes a single AUC value: it calls `collect_scores` from `evaluate_roc.py`, passing `t_fixed` through to `score_cfm_loss`, and evaluates `roc_auc_score` on the resulting scores and labels. The random seed is reset before each call, so the same draws of ε are used for every t and the comparison across the grid is paired. This does not extend to the averaged variant, which additionally draws T and therefore consumes the random stream differently.

`sweep_time` first evaluates the averaged score as a reference and then loops over the grid `T_GRID`, writing the results to `results_time_sweep.json`. `plot_sweep` reads that file and plots AUC against t , with the averaged value as a horizontal line.

The raw score values cannot be compared across different times t . For fixed y and t , the map $x \mapsto \xi(y, t \mid x)$ is affine,

$$\xi(y, t \mid x) = \left(1 - \frac{t(\sigma - 1)}{\sigma_t}\right) x + \frac{\sigma - 1}{\sigma_t} y = \frac{1}{\sigma_t} x + \frac{\sigma - 1}{\sigma_t} y, \quad (11)$$

since x enters (8) only linearly and $\sigma_t - t(\sigma - 1) = (1 - t) + t\sigma - t\sigma + t = 1$. The slope $1/\sigma_t$ is a nonzero scalar, so (11) can be solved for x , giving the conditioning point implied by the network output,

$$\hat{x}(y, t) = \sigma_t \xi(y, t) - (\sigma - 1) y,$$

and taking the difference of two instances of (11) yields:

$$\|\xi_\theta(y, t) - \xi(y, t \mid x)\|_2^2 = \frac{1}{\sigma_t^2} \|\hat{x}(y, t) - x\|_2^2.$$

The score is thus a squared error in data space, amplified by $1/\sigma_t^2$. At fixed t this factor is the same constant for every image, hence a strictly increasing rescaling of the score, which leaves the ranking and therefore the AUC unchanged.

4 Results

All results below use the configuration collected in `config.py`, listed in Table 1.

Parameter	Value
σ	0.1
Base channels	32
Batch size	64
Epochs	60
Learning rate	$3 \cdot 10^{-4}$
Monte Carlo samples for S_{CFM}	1000
RK2 steps for S_{NLL}	100
Hutchinson probes per step	5

Table 1: Configuration used for the reported results.

On a test set of 300 digit-0 images and 100 images from each of the nine remaining digit classes, the CFM score reaches

$$\text{AUC}(S_{\text{CFM}}) = 0.985.$$

The likelihood score does not separate the two classes: across four runs its AUC lies between 0.46 and 0.60, i.e. around chance level (what happens here exactly?). The remaining results all use S_{CFM} .

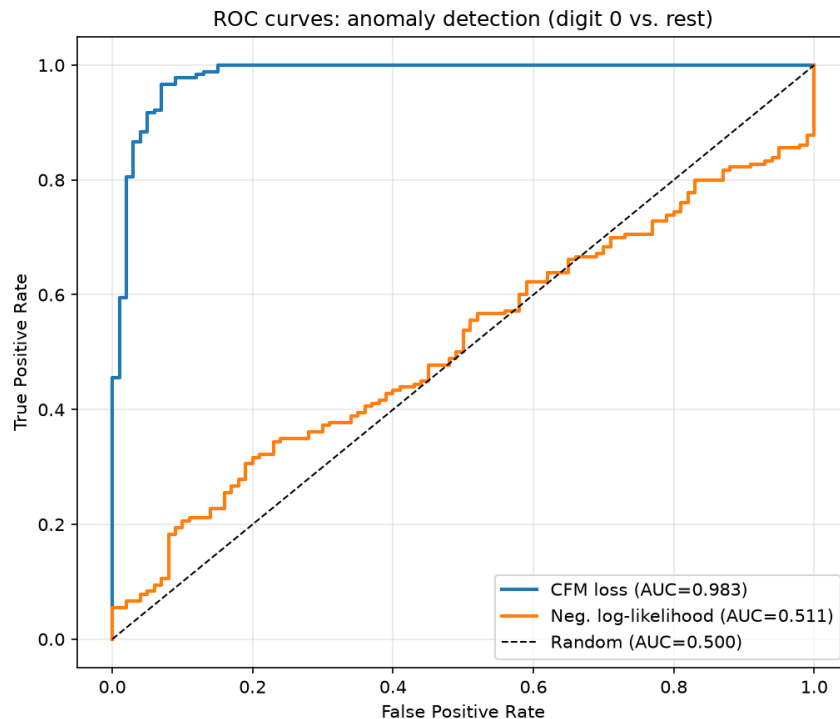


Figure 3: ROC curves for the two anomaly scores, digit 0 against all remaining digits. The CFM-based score substantially outperforms the negative log-likelihood score on this test set.

4.1 Score distributions and thresholds

Table 2 lists the score distribution per digit class over 100 test images each.

Digit	mean	median	min	max
0	0.162	0.157	0.112	0.247
1	0.207	0.205	0.178	0.258
2	0.264	0.262	0.204	0.332
3	0.272	0.268	0.213	0.369
4	0.269	0.269	0.213	0.341
5	0.262	0.260	0.198	0.322
6	0.237	0.236	0.174	0.343
7	0.255	0.249	0.185	0.341
8	0.269	0.264	0.198	0.362
9	0.257	0.255	0.208	0.322

Table 2: S_{CFM} per digit class.

Separation is weakest for digit 1, which has the lowest mean among the anomaly classes. Choosing the threshold as a quantile of the digit-0 score distribution gives the trade-off in Table 3.

Quantile	Threshold	False negatives
1.00 (max)	0.2468	42.6%
0.99	0.2461	41.9%
0.95	0.2104	9.1%
0.90	0.1954	3.0%

Table 3: Threshold choice and resulting false negative rate.

The jump between the 0.99 and 0.95 quantiles is caused by just a few atypical digit-0 images scoring 0.2468, well above the class median of 0.157.

4.2 Dependence on the smoothing parameter

One model was trained per value of σ , with all other settings fixed, and each was evaluated with the matching σ in the score.

σ	Final training loss	AUC(S_{CFM})
0.01	0.187	0.945
0.05	0.175	0.983
0.10	0.165	0.982
0.20	0.151	0.981

Table 4: Effect of the smoothing parameter σ .

4.3 Number of Monte Carlo samples

Since S_{CFM} is itself a Monte Carlo estimate, its variance depends on the number of draws. The same trained model was evaluated at several sample counts.

num.samples	AUC(S_{CFM})
100	0.975
500	0.982
1000	0.985
4000	0.985

Table 5: Effect of the number of Monte Carlo draws in the score.

The estimate saturates at around 1000 draws, which is why this value is used throughout.

4.4 Spatial decomposition of the score

Since S_{CFM} averages the squared mismatch over the d output dimensions, the mismatch can also be kept per dimension,

$$H(x)_k = \mathbb{E}_{T,\varepsilon} \left[(\xi_{\theta^*}(Y, T) - \xi(Y, T \mid x))_k^2 \right], \quad k = 1, \dots, d,$$

so that $S_{\text{CFM}}(x) = \frac{1}{d} \sum_k H(x)_k$. Reshaping $H(x)$ to 28×28 gives a heatmap showing where in the image the learned field fails to reproduce the conditional target, providing a spatial diagnostic of where the learned field disagrees most strongly with the observation. Figures 4 and 5 show this for digit 1 and digit 4 respectively, in each case alongside four digit-0 images and on a color scale shared within the figure.

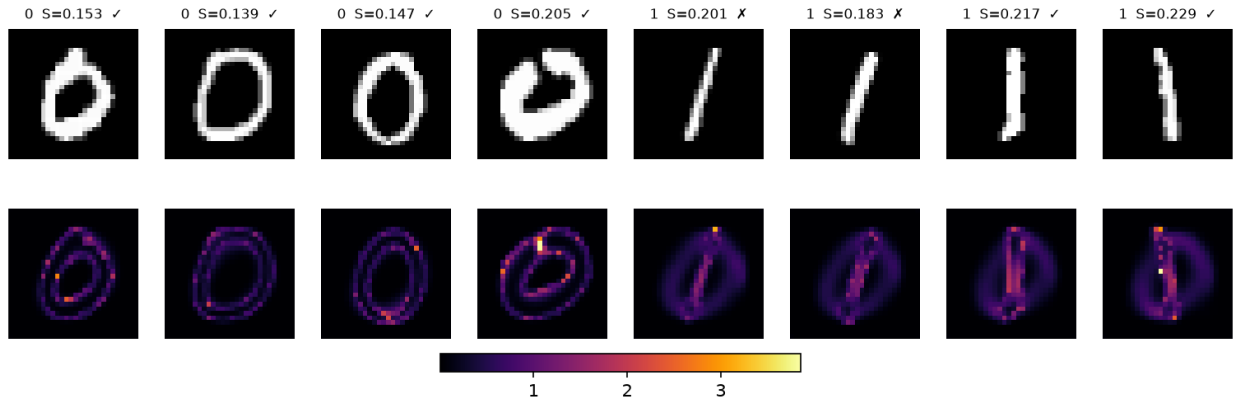


Figure 4: Per-pixel decomposition of S_{CFM} for digit 0 against digit 1. Top: input images. Bottom: contribution of each pixel to the score, on a shared color scale. Titles give the true class, the score, and whether the image is classified correctly at the threshold $\tau = 0.2104$ (the 0.95 quantile of the digit-0 scores).

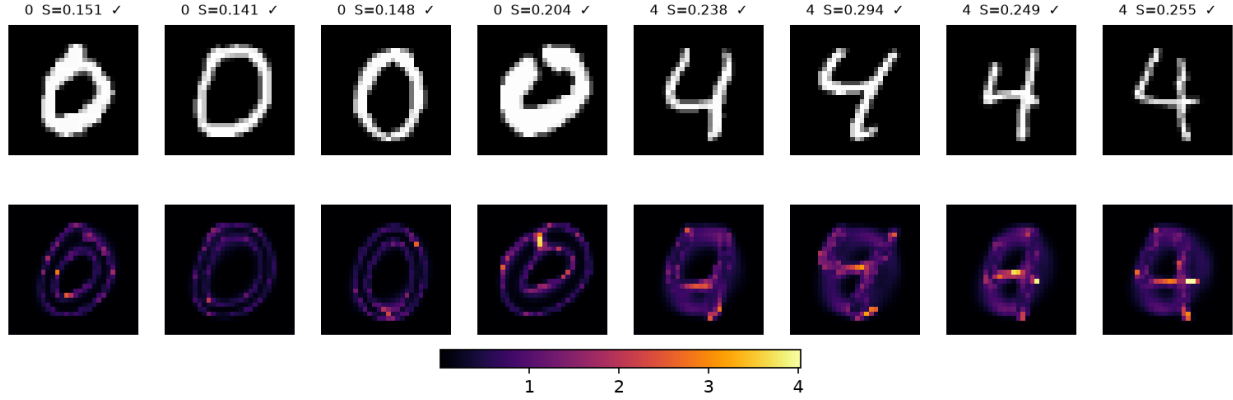


Figure 5: The same decomposition for digit 0 against digit 4.

4.5 Dependence on the time variable

S_{CFM} averages over $T \sim U([0, 1])$. Evaluating it at fixed t instead, with the same trained model and the same test set, gives Figure 6 and Table 6.

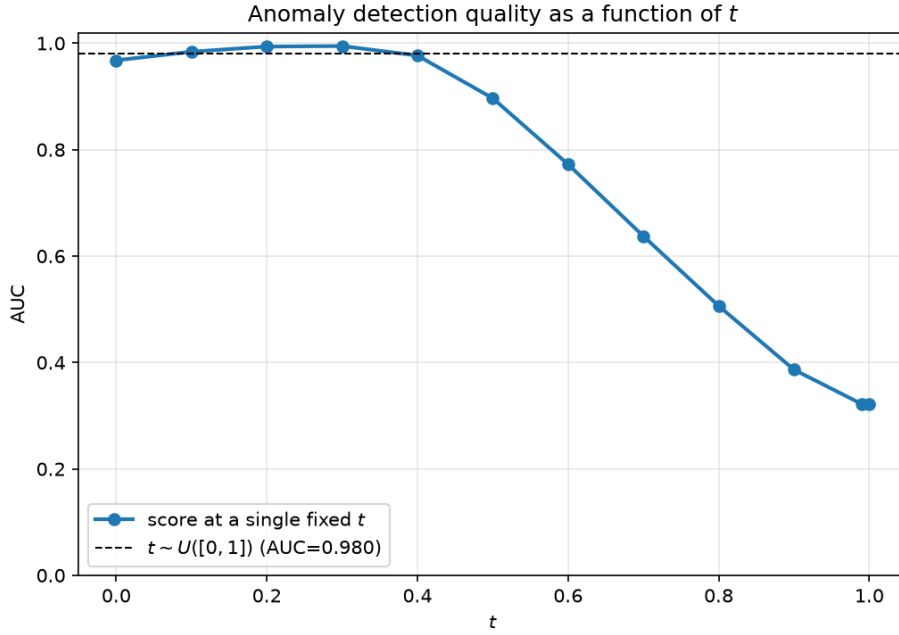


Figure 6: AUC of S_{CFM} evaluated at a single fixed t , against t . The dashed line is the averaged score used elsewhere in this document.

t	$\text{AUC}(S_{\text{CFM}})$
0.00	0.968
0.10	0.984
0.20	0.994
0.30	0.995
0.40	0.977
0.50	0.897
0.60	0.772
0.70	0.638
0.80	0.506
0.90	0.386
0.99	0.322
1.00	0.321
$T \sim U([0, 1])$	0.985

Table 6: AUC at fixed t , and the averaged score for comparison.

Evaluating S_{CFM} at a fixed time t reveals that the optimal anomaly separation is concentrated in a narrow window around $t \in [0.2, 0.3]$ ($\text{AUC} \approx 0.995$). Beyond $t \approx 0.5$, performance degrades significantly—even reversing the ranking near $t = 1$ ($\text{AUC} = 0.321$)—which explains why time-averaging over $T \sim U([0, 1])$ yields a slightly lower overall score ($\text{AUC} = 0.980$).

At $t = 0$, the score simplifies to a baseline distance metric. Since $Y_0 = \varepsilon$ contains no information about the input x , a network minimizing the mean squared error predicts the conditional expectation of the target $\xi(y, 0 \mid x) = x + (\sigma - 1)y$. This yields $\xi_{\theta}(y, 0) = \mu_0 + (\sigma - 1)y$, where μ_0 is the mean of the normal training samples.

The noise term cancels in the residual $\mu_0 - x$, reducing the score to:

$$S_{\text{CFM}}(x)|_{t=0} = \frac{1}{d} \|x - \mu_0\|_2^2.$$

This score reflects the theoretical minimizer of the objective rather than a direct measurement, as $t = 0$ is a set of measure zero during training. Nevertheless, it aligns well with the observed $\text{AUC} = 0.968$, showing that a large portion of the separation is achieved by simple Euclidean distance to the mean, while the vector field at $t > 0$ yields additional useful information beyond the $t = 0$ baseline.

An important observation is that the "successful" anomaly score does not use the generative structure of the learned flow. Once ξ_{θ^*} has been trained, S_{CFM} can be evaluated without solving the ODE or estimating a density. The generative structure was therefore essential for the original flow model and for the likelihood experiment, but is not required by S_{CFM} . Maybe a substantially simpler model could achieve comparable performance...